

实验 6 动态分区分配

6.1 实验目的

- (1) 理解动态分区分配方式中使用的数据结构;
- (2) 理解动态分区存储管理方式;
- (3) 掌握动态分区分配算法的实现方法;

6.2 预备知识

1.首次适应算法

算法思想：每次都从低地址开始查找，找到第一个能满足大小的空闲分区，优先使用低地址空间。

如何实现：空闲分区以地址递增的次序排列。每次分配内存时顺序查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。

2.最佳适应算法

算法思想：由于动态分区分配是一种连续分配方式，为各进程分配的空间必须是连续的一整片区域。因此为了保证当“大进程”到来时能有连续的大片空间，可以尽可能多地留下大片的空闲区，即，优先使用更小的空闲区。

如何实现：空闲分区按容量递增次序链接。每次分配内存时顺序查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。

缺点：每次都选最小的分区进行分配，会留下越来越多的、很小的、难以利用的内存块。因此这种方法会产生很多的外部碎片。

3.最坏适应算法

最坏适应算法又称最大适应算法。

算法思想：为了解决最佳适应算法的问题---即留下太多难以利用的小碎片，可以在每次分配时优先使用最大的连续空闲区，这样分配后剩余的空闲区就不会太小，更方便使用。

如何实现：空闲分区按容量递减次序链接。每次分配内存时顺序查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。

缺点：每次都使用最大的连续空闲区会导致大空闲区被很快用光，后续再有大进程到来时就不会再有可用的大分区了。

4. 邻近适应算法

算法思想：首次适应算法每次都从链头开始查找的。这可能会导致低地址部分出现很多小的空闲分区，而每次分配查找时，都要经过这些分区，因此也增加了查找的开销。如果每次都从上次查找结束的位置开始检索，就能解决上述问题。

如何实现：空闲分区以地址递增的顺序排列（可排成一个循环链表）。每次分配内存时从上次查找结束的位置开始查找空闲分区链（或空闲分区表），找到大小能满足要求的第一个空闲分区。

6.3 实验内容

(1)用 C 语言实现采用首次适应算法(地址从小到大)的动态分区分配过程 `alloc()` 和回收过程 `Free()`(即自己编写一个分区分配和释放的函数)。其中,空闲分区通过空闲分区链来管理;在进行内存分配时,系统优先使用空闲区低端的空间。

(2)假设初始状态下,可用的内存空间为 640 KB(即程序开始运行时,用 `malloc()`函数一次申请 640KB 的内存空间,程序结束时用 `free()`函数释放空间。然后自己编写程序在这 640KB 的空间上模拟首次适应算法分配过程。自己编写的分配过程不是真正去向操作系统申请空间,而是在这 640KB 空间上标记哪个块占用了;释放过程也不是真正释放空间,只是标记哪块分区空闲),作业请求序列如下:

- 作业 1 申请 130 KB
- 作业 2 申请 60 KB
- 作业 3 申请 100 KB
- 作业 2 释放 60 KB
- 作业 4 申请 200 KB
- 作业 3 释放 100 KB
- 作业 1 释放 130 KB
- 作业 5 申请 140 KB
- 作业 6 申请 60 KB
- 作业 7 申请 50 KB
- 作业 6 释放 60 KB

请采用首次适应算法进行内存块的分配和回收,要求每次分配和回收后显示出空闲内存分区链的情况。

6.4 实验指导

1.内存分配

从链首开始顺序查找,直至找到一个大小能满足要求的空闲分区为止,然后再按照作业的大小,从该分区中划出一块内存空间分配给请求者,余下的空闲分区仍留在空闲链中。

若从链首直至链尾都不能找到一个能满足要求的分区,则此次内存分配失败,返回。

在采用首次适应算法的动态分区分配过程 `alloc()`和回收过程 `free()`时,空闲分区通过空闲分区链表来管理,系统优先使用空闲区低端的空间,即每次分配内存空间是总是从低址部分开始进行循环,找到第一个合适的空间,便按作业所需分配的大小分配给作业。

2.内存回收

将释放作业所在内存块的状态改为空闲状态,删除其作业名,设置为空。并判断该空闲块是否与其他空闲块相连,若释放的内存空间与空闲块相连时,则合并为同一个空闲块,同时修改分区大小及起始地址。

此时要考虑到四种情况:

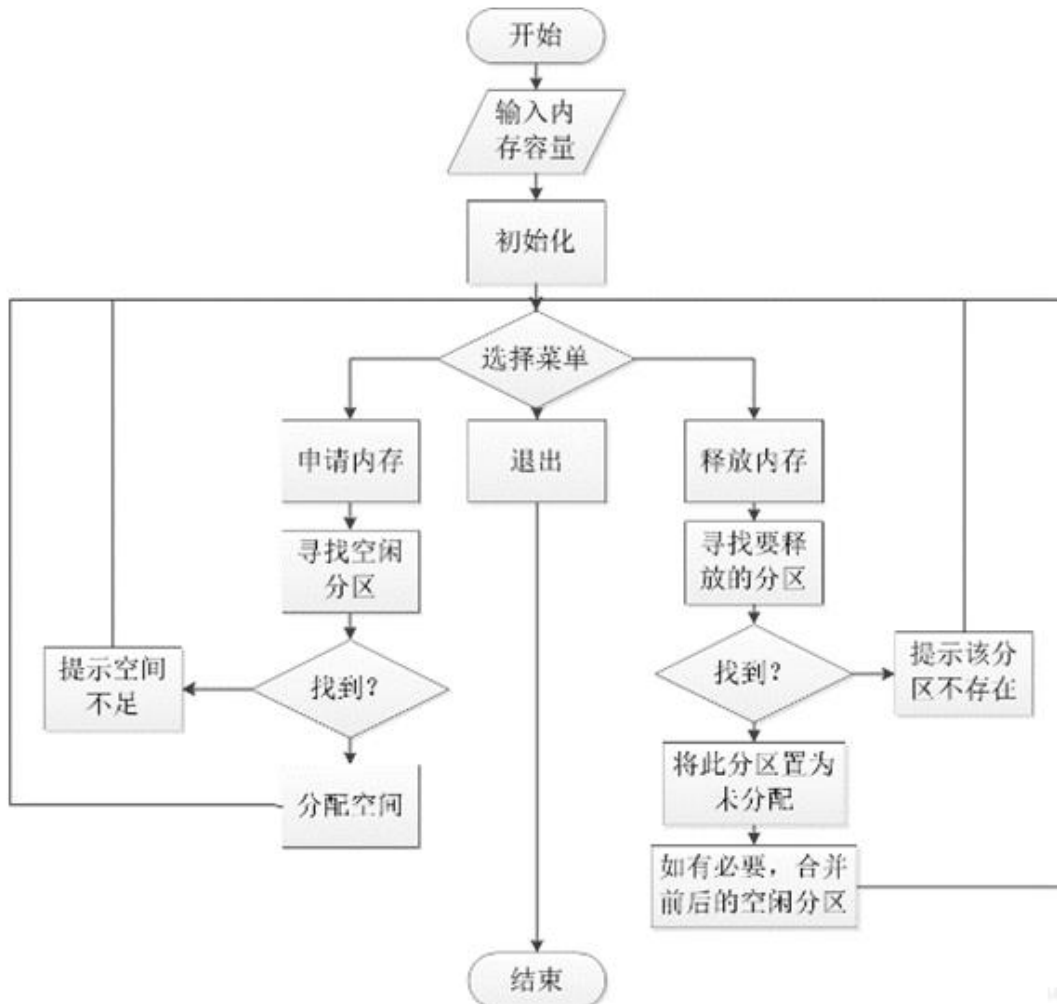
(1) 回收区与插入点的前一个空闲分区相邻接。此时将二者合并,修改前一分区的大小。

(2) 回收区与插入点的后一空闲分区相邻接,将二者合并,用回收区的首址作为新空闲区的首址。

(3) 回收区同时与插入点的前后两个空闲分区相邻接，三者合并，使用前一空闲分区的表项和首址，并修改其大小。

(4) 回收区单独存在。

3.流程图



4. 函数设计

(1) void initNode(struct nodespace *p)

创建一个双链表存储信息

(2) void myMalloc1(int teskid, int size, struct nodespace *node)

申请空间函数

(3) void myFree(int teskid, struct nodespace *node)

释放空间函数

(4) void printNode(struct nodespace *node)

打印输出节点存储信息了，即内存申请剩余情况

(5) void destory(struct nodespace *node)

退出程序并销毁清空节点

(6)void menu()

主菜单，提示用户进行相应的操作

6.5 参考源代码

```
#include<stdio.h>
#include<stdlib.h>
typedef struct Node
{
    int jobid;    // 作业号
    int begin;    // 开始地址
    int size;     // 大小
    int status;   // 状态 0 代表占用，1 代表空闲
    struct Node *next; // 后指针
}Node;

void initNode(Node *p)
{
    if(p == NULL) //如果为空则新创建一个
    {
        p = (Node*)malloc(sizeof(Node));
    }
    p->jobid = -1;
    p->begin = 0;
    p->size = 640;
    p->status = 1;
    p->next = NULL;
}
```

```

void alloc(int jobid,int size,Node *node)
{
    while(node != NULL)
    {
        if(node->status == 1)    //空闲的空间
        {
            if(node->size > size)    //当需求小于剩余空间充足的情况
            {
                //分配后剩余的空间
                Node *p = (Node*)malloc(sizeof(Node));
                p->begin = node->begin + size;
                p->size = node->size - size;
                p->status = 1;
                p->jobid = -1;
                //分配的空间
                node->jobid = jobid;
                node->size = size;
                node->status = 0;
                //改变节点的连接
                p->next = node->next;
                node->next = p;
                printf("=====================分配内存成功! =====\n");
                break;
            }
            else if(node->size == size)    //需求空间和空闲空间大小相等时
            {
                node->jobid = jobid;
                node->size = size;
                node->status = 0;
                printf("=====================分配内存成功! =====\n");
                break;
            }
        }
        if(node->next == NULL)
        {
            printf("=====================分配失败, 没有足够的空间! =====\n");
            break;
        }
        node = node->next;
    }
}

```

```

void Free(int jobid,Node *node)
{
    if(node->next == NULL && node->jobid == -1)
    { printf("=====您还没有分配任何作业！ =====\n");
    }
    while(node != NULL)
    { if(node->status == 1 && node->next->status == 0 && node->next->jobid == jobid) //释
    放空间的上一块空间空闲时
        {
            node->size = node->size + node->next->size;
            Node *q = node->next;
            node->next = node->next->next;
            free(q);
            printf("=====释放内存成功！ =====\n");
            if(node->next->status == 1) //下一个空间是空闲空间时
            {node->size = node->size + node->next->size;
            Node *q = node->next;
            node->next = node->next->next;
            free(q);
            printf("=====释放内存成功！ =====\n");
            }
            break;
        }
    else if(node->status == 0 && node->jobid == jobid) //释放空间和空闲空间不连续时
    {node->status = 1;
    node->jobid = -1;
    if(node->next != NULL && node->next->status == 1) //下一个空间是空闲空间时
        {
            node->size = node->size + node->next->size;
            Node *q = node->next;
            node->next = node->next->next;
            free(q);
        }
        printf("=====释放内存成功！ =====\n");
        break;
    }
    else if(node->next == NULL) //作业号不匹配时
    {printf("=====没有此作业！ =====\n");
    break;
    }
    node = node->next;
}
}

```

```

void printNode(Node *node)
{
    printf("                内存情况                \n");
    printf("-----\n");
    printf("| 起始地址\t结束地址\t大小\t状态\t作业号\t\n");
    while(node != NULL)
    {
        if(node->status==1)
        {
            printf("| %d\t%d\t%dKB\tfree\t无\t\n", node->begin + 1, node->begin+node->size,
node->size);
        }
        else
        {
            printf("| %d\t%d\t%dKB\tbusy\t%d\t\n", node->begin + 1, node->begin+node->size,
node->size, node->jobid);
        }
        node = node->next;
    }
    printf("-----\n");
}

void destory(Node *node)
{
    Node *q = node;
    while(node != NULL)
    {
        node = node->next;
        free(q);
        q = node;
    }
}

```

```

int main()
{
    Node *init = (Node*)malloc(sizeof(Node));
    Node *node = NULL;
    initNode(init); //初始化主链
    node = init;    //指向链表头
    int op; int jobid; int size;
    while(1)
    {
        menu(); //显示主菜单
        scanf("%d",&op);
        switch(op)
        {
            case 1:
                printf("请输入作业号: ");
                scanf("%d",&jobid);
                printf("此作业申请的空间大小(KB):");
                scanf("%d",&size);
                alloc(jobid,size,node);
                printf("\n");
                printNode(node);
                break;
            case 2:
                printf("请输入作业号:");
                scanf("%d",&jobid);
                Free(jobid,node);
                printf("\n");
                printNode(node);
                break;
            case 3:
                printNode(node);
                break;
            case 4:
                destory(node);
                initNode(init);
                node = init;
                exit(0);
            default:
                printf("=====您的输入有误, 请重新输入! =====\n");
                continue;
        }
    }
    return 0;
}

```


[illegible]